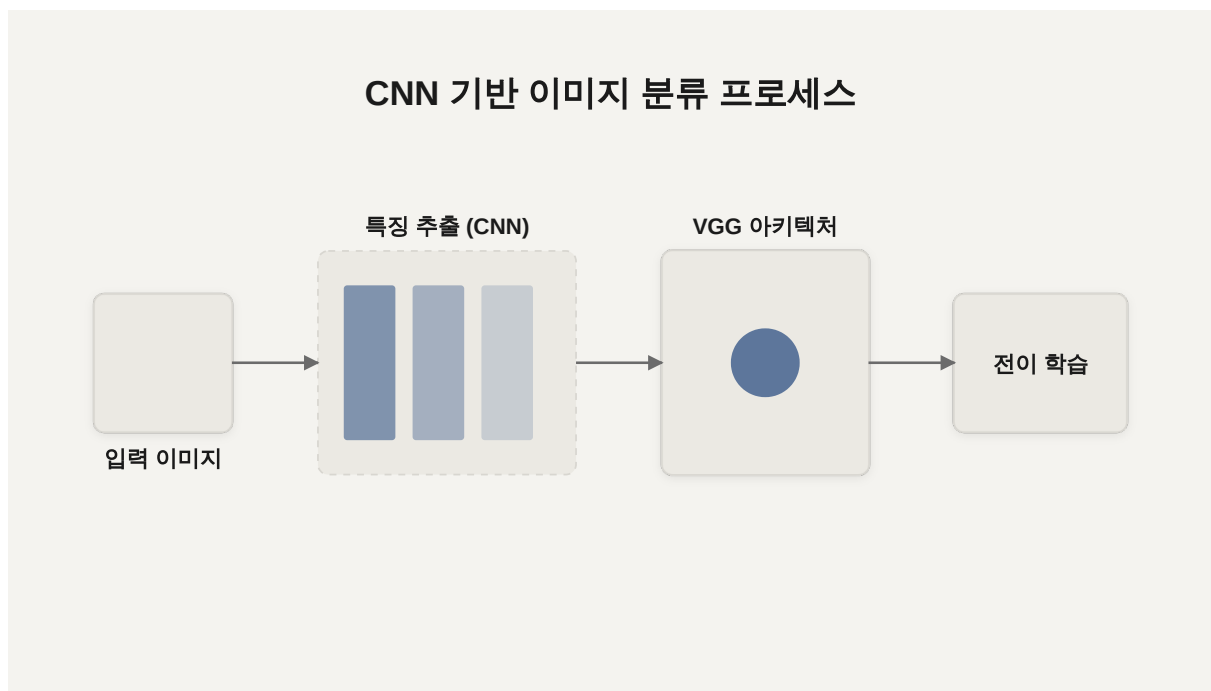


STEP 2. CNN 개념 (2~3일)

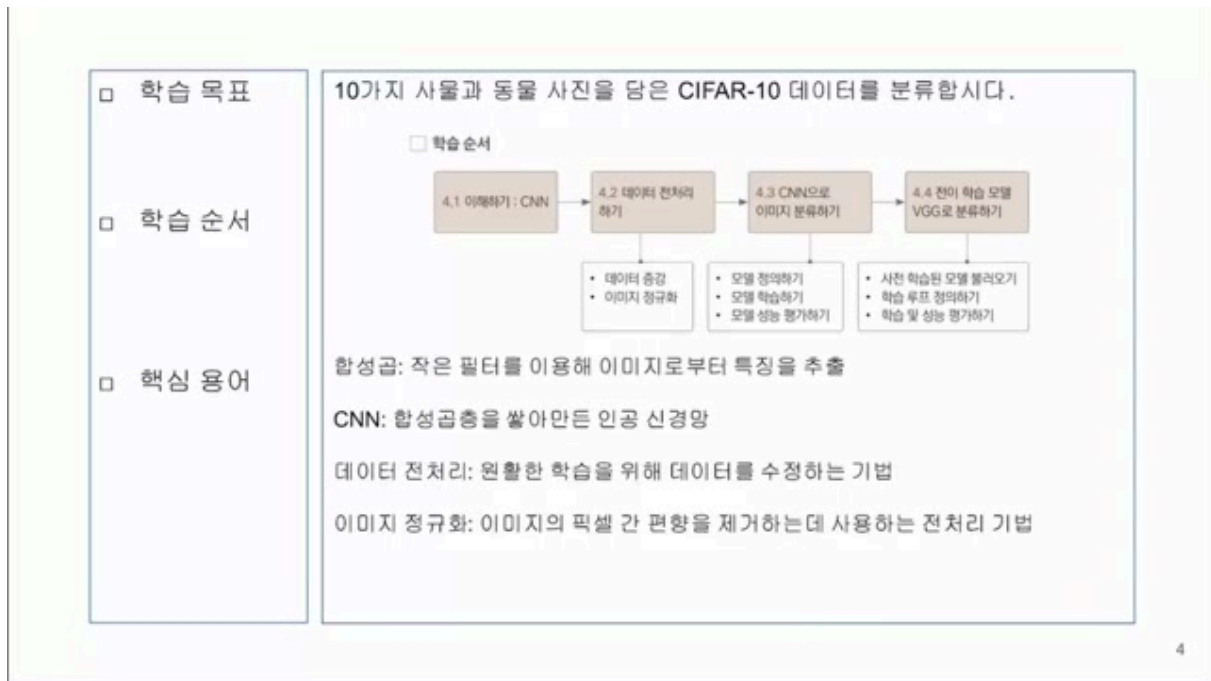
CNN과 VGG 모델로 사진을 분류하는 방법은 무엇인가? CNN은 합성곱층을 통해 이미지 특징을 추출하고, VGG는 사전 학습된 모델을 활용하여 적은 데이터로도 높은 정확도를 달성할 수 있습니다.

1. CNN과 VGG로 사진 분류하기

이번 주는 사진 분류를 위한 두 가지 새로운 알고리즘인 CNN과 VGG에 대해 학습한다.



1.1. CNN (Convolutional Neural Network) 이해하기



CNN은 합성곱층을 통해 이미지의 특징을 추출하는 인공 신경망이다.

1. 학습 목표

- 10가지 사물과 동물 사진으로 구성된 CIFAR-10 데이터셋을 CNN으로 분류한다.
- CNN의 개념을 이해하고, 데이터 전처리 과정을 학습한다.
- CNN으로 직접 이미지를 분류하고, 전이 학습의 개념을 배운다.

2. 주요 키워드

- **합성곱 (Convolution):** 작은 필터를 사용하여 이미지로부터 특징을 추출하는 과정이다.
- **데이터 전처리 (Data Preprocessing):** 모델의 입력이 되기 전에 데이터를 수정하는 과정으로, 원활한 학습을 위해 데이터를 가공한다.
- **이미지 정규화 (Image Normalization):** 데이터 전처리 기법 중 하나로, 이미지의 색상 편향을 제거하는 데 사용된다.

3. MLP의 한계와 합성곱의 필요성

- 이전에는 MLP(Multi-Layer Perceptron)를 사용하여 이미지를 분류했다.
- MLP는 이미지의 각 픽셀 위치에 대한 가중치를 학습하기 때문에, 특징의 크기 변화에 민감하다.
 - 예를 들어, 동그라미 특징을 인식하는 MLP 모델이 있을 때, 동그라미의 크기가 커지거나 작아지면 이를 인식하지 못하게 된다.

- 이는 각 픽셀의 위치에 대한 의존성 때문에 발생하며, 특정 위치에 특징이 없으면 해당 특징이 아니라고 판단하게 된다.
- 이러한 MLP의 특성은 과적합(Overfitting)에 취약하게 만든다.
- 과적합을 피하기 위한 드롭아웃과 같은 기법들은 성능을 의도적으로 저하시키는 경우가 많았다.
- 합성곱은 이미지의 모든 픽셀을 보는 대신, 작은 영역(필터)을 사용하여 특징을 추출함으로써 이러한 한계를 극복한다.

4. 합성곱의 작동 방식

- 합성곱은 작은 영역(필터)을 이미지 위에서 조금씩 이동시켜 가며 특징을 추출한다.
- **컨볼루션 (Convolution)**: 이미지로부터 특징을 추출하기 위한 가중치를 행렬로 나타낸 것이다.
 - 일반적으로 3x3 행렬 형태의 필터(커널)를 사용한다.
- **필터 (Filter)**: 여러 개의 커널(Kernel)의 모임이다.
- **커널 (Kernel)**: 하나의 채널에 대한 특징을 추출한다.
 - 이미지는 RGB 세 개의 채널로 구성되어 있으므로, 각 채널에 대응하는 커널이 필요하다.
 - 이러한 커널들이 모여 하나의 필터가 된다.

5. 합성곱 관련 용어

- **특징 맵 (Feature Map)**: 합성곱 연산의 결과로 얻어지는 이미지로, 원본 이미지에서 추출된 특정 특징만을 담고 있다.
- **스트라이드 (Stride)**: 커널이 이미지 위에서 이동하는 거리를 나타낸다.
 - 예를 들어, 중심이 한 칸씩 이동하는 것을 스트라이드라고 한다.

6. CNN의 전체적인 동작 과정

- 입력 이미지는 여러 단계의 합성곱을 거치면서 점점 더 세분화된 특징을 추출한다.
 - 초기 단계에서는 가지, 이파리, 사과 모양, 빛 반사 등 비교적 큰 특징을 추출한다.
 - 이후 단계에서는 추출된 특징의 일부분(가지의 일부분, 사과의 동그라미 일부분 등)을 다시 추출하여 더 세밀한 특징을 얻는다.
- 추출된 세세한 특징들은 1차원 벡터로 변환된다.
- 이 벡터는 MLP 분류기를 거쳐 각 클래스에 해당할 확률을 계산한다.

- 가장 높은 확률을 가진 클래스로 이미지를 분류한다.
- 실제로는 다양한 특징을 추출하는 여러 개의 필터가 사용된다.

7. CNN 기본 블록 구성

- CNN의 기본 블록은 합성곱(Convolution), 활성화 함수(ReLU), 합성곱(Convolution), 활성화 함수(ReLU), 맥스 풀링(Max Pooling) 순서로 구성된다.
- **렐루 (ReLU, Rectified Linear Unit) 함수:**
 - 기울기 소실 문제(Vanishing Gradient Problem)를 완화하기 위해 사용된다.
 - 기울기 소실 문제는 신경망의 층이 깊어질수록 기울기가 점점 0에 가까워져 학습이 어려워지는 현상이다.
 - ReLU 함수는 0보다 작은 입력은 0으로, 0보다 큰 입력은 그대로 출력하는 함수이다.
 - 이 함수의 미분값은 0 또는 1이므로, 기울기 소실 문제를 어느 정도 방지할 수 있다.
- **맥스 풀링 (Max Pooling):** 이미지의 크기를 줄이는 알고리즘 중 하나이다. (자세한 내용은 아래 66-73에서 설명)

8. 맥스 풀링 (Max Pooling) 상세 설명

- 맥스 풀링은 입력 이미지를 여러 구역으로 나누고, 각 구역에서 최대값을 선택하여 새로운 이미지를 생성한다.
- 예를 들어, 4개 픽셀 구역에서 가장 큰 값(예: 4)을 선택한다.
- 이 과정을 통해 이미지의 크기가 줄어들고 정보 손실이 발생하지만, 이는 과적합을 방지하기 위한 중요한 수단이다.

9. 패딩 (Padding) 기법

- 패딩은 합성곱 연산 시 이미지 크기가 점점 작아지는 것을 방지하기 위해 사용된다.
- 입력 이미지 주변에 0으로 값을 채워(패딩) 입력 크기와 출력 크기를 같게 유지할 수 있다.
- 이를 통해 합성곱을 여러 번 적용해도 이미지 크기가 유지되어 더 깊은 신경망을 구성할 수 있다.
- 패딩은 가중치에 대한 영향도 줄여 결과적으로 입력과 출력의 크기를 동일하게 유지하면서 합성곱을 적용하는 것을 가능하게 한다.

10. CNN 모델 코드 구현

- 파이토치(PyTorch)를 사용하여 신경망 모듈을 정의하고 클래스를 생성한다.
- **기본 블록 (Basic Block):**
 - 입력 채널 수(in_channels), 출력 채널 수(out_channels), 히든 레이어 수(hidden_dim)를 설정한다.
 - 순서대로 합성곱, 활성화 함수(ReLU), 합성곱, 활성화 함수(ReLU), 맥스 풀링을 적용한다.
- **CNN 모델 정의:**
 - 32×32 크기의 이미지가 입력으로 들어온다.
 - 기본 블록을 여러 번 거치면서 이미지 크기는 절반으로 줄어들고 채널 수는 증가한다.
 - 최종적으로 1차원 벡터로 평탄화(Flatten) 과정을 거친다.
 - CNN은 2차원 이미지를 입력으로 받지만, MLP는 1차원 벡터만 입력으로 받기 때문에 이 과정이 필요하다.
 - 평탄화 층은 이미지를 1차원 벡터로 변환하여 MLP 분류기에 입력으로 사용한다.
 - 평탄화된 벡터는 분류기(MLP)를 거쳐 클래스 확률 분포를 출력한다.
- **분류기 (Classifier):**
 - 평탄화된 벡터를 입력받아 최종 클래스 개수(예: 10개)로 줄여준다.
 - MLP 구조로 활성화 함수와 드롭아웃을 포함할 수 있다.
- **학습 루프 (Training Loop):**
 - 모델 정의, 데이터 로드, 데이터 증강, 전처리, 손실 계산, 역전파, 최적화 순서로 진행된다.

1.2. 데이터 전처리 및 학습 과정

4.4 모델 학습하기

▼ CNN 모델의 학습 루프



Must Have 텐초의 파이토치 딥러닝 특강

1. 데이터 전처리 상세

- **컴포즈 (Compose)**: 여러 전처리 기법들을 순서대로 적용한다.
- **랜덤 크롭 (Random Crop)**: 이미지의 일부분만 선택하여 사용한다.
- **패딩 (Padding)**: 이미지 주변을 0으로 채워 크기를 조절한다.
 - `padding=4` 는 이미지 주변에 4픽셀만큼 패딩을 적용하라는 의미이다.
- **랜덤 호리젠탈 플립 (Random Horizontal Flip)**: 이미지를 좌우 대칭으로 뒤집는다.
- **텐서로 변환 (To Tensor)**: 파이썬 이미지를 파이토치 텐서로 변환한다.
- **정규화 (Normalization)**:
 - 데이터의 분포를 정규분포(가우스 분포)를 따르도록 평균과 표준편차를 사용하여 조정한다.
 - 이미지넷과 같은 대규모 데이터셋의 평균 및 표준편차를 사용하기도 한다.
 - 이미지 픽셀 값의 편향(예: 특정 색상이 많은 경우)을 제거하여 모델이 더 잘 학습하도록 돕는다.

2. 데이터 로드 및 모델 정의

- 데이터셋을 로드하고, 배치 크기, 학습/테스트 분할 등을 설정한다.
- GPU 사용 여부를 결정하고 모델을 GPU로 보낸다.

3. 학습 루프 상세

- 학습률(Learning Rate)과 최적화 함수(Optimizer)를 설정한다.
- 정해진 에포크(Epoch)만큼 반복 학습을 진행한다.
- 각 에포크마다 데이터를 로드하고, 손실을 계산하며, 역전파 및 최적화를 수행한다.
- 일정 에포크마다 학습 상태(에포크, 손실)를 출력하고, 모델 가중치를 저장한다.

4. 성능 평가

- 학습된 모델을 사용하여 테스트 데이터셋에 대한 정확도를 평가한다.
- 기울기 사용을 비활성화하고, 모델 예측값과 실제 정답을 비교하여 정확도를 계산한다.
- 이전 학습 결과 약 83%의 정확도를 얻었다.

2. VGG 모델을 이용한 전이 학습

전이 학습은 대량의 데이터로 학습된 모델을 가져와 우리가 가진 데이터에 맞게 최적화하는 방법이다.

2.1. VGG 모델 소개

4.5 모델 성능 평가하기

```

# 모델 성능 확인
model.load_state_dict(torch.load("CIFAR.pth", map_location=device))
num_corr = 0
with torch.no_grad():
    for data, label in test_loader:
        output = model(data.to(device))
        preds = output.data.max(1)[1]
        corr = preds.eq(label.to(device).data).sum().item()
        num_corr += corr
    print(f"Accuracy: {num_corr / len(test_data)}")
Accuracy: 0.8354
83%

```

Must Have 텐초의 파이토치 딥러닝 특강

1. VGG 모델이란?

- VGG는 우리가 직접 만든 CNN 모델의 대표적인 예시이다.

- 224×224 크기의 이미지를 입력받아 여러 합성곱을 거쳐 최종적으로 1000개의 클래스를 분류하는 구조를 가진다.
 - 이는 ImageNet 데이터셋을 사용했기 때문이다.

2. 전이 학습의 원리

- 대규모 데이터셋으로 미리 학습된 모델은 기본적인 특징 추출 능력이 뛰어나다.
- 이는 마치 수능 공부를 위해 과거 기출문제를 모두 풀어본 친구에게 팁을 얻는 것과 같다.
- 이러한 사전 학습된 모델을 활용하면, 적은 데이터로도 높은 정확도를 달성할 수 있으며 학습 효율성이 높아진다.
- 일반적으로 대용량 데이터로 한번 학습한 후, 보유한 작은 데이터로 최적화하는 방식을 사용한다.

3. 사전 학습된 VGG 모델 불러오기

- `torchvision` 라이브러리에서 VGG 모델을 제공하므로 이를 활용한다.
- `pretrained=True` 옵션을 사용하여 사전 학습된 가중치를 불러온다. `pretrained=False` 로 설정하면 랜덤하게 초기화된 가중치를 사용한다.
- 사전 학습된 VGG 모델은 1000개의 클래스를 분류하도록 학습되었으므로, 우리가 사용할 10개 클래스에 맞게 분류기를 새로 정의해야 한다.
- **분류기 재정의:**
 - 기존 모델의 마지막 부분을 제거하고, 새로운 MLP 구조로 대체한다.
 - MLP 층, 활성화 함수(ReLU), 드롭아웃(Dropout)을 포함하여 과적합을 방지한다.
 - 최종적으로 10개의 클래스를 출력하도록 설계한다.
- VGG 모델의 분류기를 새로 정의한 분류기로 교체한다.
- 모델을 GPU로 보내 학습 준비를 마친다.

4. 전이 학습을 위한 전처리

- VGG 모델은 224x224 크기의 이미지를 입력으로 받는다.
- 우리의 데이터셋(CIFAR-10)은 32x32 크기이므로, `리사이징(Resizing)` 을 통해 크기를 맞춰야 한다.
- 이후 랜덤 크롭, 좌우 대칭, 텐서 변환, 정규화 과정을 거친다.

5. 전이 학습 학습 루프

- 데이터셋 및 데이터 로더를 준비한다.
- 각 스텝마다 최적화 함수를 설정하고, 약 30번 반복 학습을 진행한다.
- 데이터 로드, 기울기 초기화, 예측값 계산, 손실 계산, 역전파, 최적화 순서로 진행된다.
- `logger` 메소드를 사용하여 에포크와 손실을 기록한다.
- 학습 완료 후 모델 가중치를 저장하고, 성능 평가를 수행한다.
- 평가 결과 약 92%의 정확도를 얻었다.

2.2. 전이 학습의 효과

4.6.4 모델 성능 평가하기

```

model.load_state_dict(torch.load("CIFAR_pretrained.pth", map_location=device))

num_corr = 0

with torch.no_grad():
    for data, label in test_loader:

        output = model(data.to(device))
        preds = output.data.max(1)[1]
        corr = preds.eq(label.to(device).data).sum().item()
        num_corr += corr

print(f"Accuracy:{num_corr/len(test_data)}")

Accuracy:0.9274

```

Must Have 텐초의 파이토치 딥러닝 특강

- 전이 학습을 통해 약 83%에서 92%로 정확도가 크게 향상되었다.
- 이는 대규모 데이터셋으로 사전 학습된 모델이 이미지의 기본적인 특징을 잘 학습하고 있기 때문이다.
- 따라서 해결하고자 하는 문제에 대해 대규모 데이터셋으로 학습된 공개 모델이 있다면, 처음부터 학습시키기보다 해당 모델을 가져와 자신의 데이터에 최적화하는 것이 훨씬 효율적이다.